

Lab 4 — Code Hint Sheet

A VAE for Spectra QC, Anomaly Detection & Generation

Stuck on a **YOUR TURN** cell? Each blank you need is below with a few options. Pick one, type it in, and **run the cell** — the built-in `assert` checks will tell you whether it is right. Only the three marked cells need editing; run everything else as-is. The encoder, decoder, and the reparameterization trick are already written for you.

Step 3 — the bottleneck (latent) size. The narrow layer every spectrum must squeeze through (Lecture 11). Keep it *small* so unfamiliar spectra can't be rebuilt — that is what makes their reconstruction error spike. Value 2 also lets us plot the latent space directly.

Blank 1 — LATENT_DIM.

- (a) `LATENT_DIM = 2`
- (b) `LATENT_DIM = 256`
- (c) `LATENT_DIM = 1`

(a) is right — small enough to force real compression, and exactly 2 so the 2-D latent plot in Step 9 works (the check requires `LATENT_DIM == 2`). (b) is **too big**: a wide bottleneck lets the network copy *any* input straight through, so even weird spectra reconstruct well and the **anomaly detector stops working**. (c) is **too tight** and can't be plotted in 2-D — both (b) and (c) fail the `== 2` check.

Step 5 — the combined VAE loss. A VAE is trained on *two terms added together* (Lecture 11's recipe slide): the `recon_loss` (rebuild the spectrum accurately) plus the `kl_loss` (keep the latent cloud tidy), with the KL weighted by `beta`. Both terms are already computed for you — write the one line that combines them.

Blank 2 — loss.

- (a) `loss = recon_loss + beta * kl_loss`
- (b) `loss = recon_loss`
- (c) `loss = recon_loss - beta * kl_loss`

(a) is the VAE objective: reconstruction *plus* the weighted KL “tidiness” term. (b) drops the KL entirely — that is just a plain autoencoder, and the check (loss must equal `recon_loss + BETA*kl_loss`) fails. (c) *subtracts* the KL, which rewards a messy latent and also fails the check — and would blow training up. (Turning `BETA` way *up* instead makes reconstructions blurry — tidiness beats accuracy.)

Step 7 — the anomaly threshold. Turn reconstruction error into a flag. Measure the error on the *normal training* spectra, then flag anything well above that. A high percentile — the **95th** — expects to false-alarm on only ~5% of normal spectra. This is a sensitivity/specificity dial (Lecture 10 callback).

Blank 3 — THRESHOLD.

- (a) `THRESHOLD = np.percentile(train_errors, 95)`
- (b) `THRESHOLD = 0`
- (c) `THRESHOLD = train_errors.max()`

(a) is the 95th-percentile cut — it lands strictly *inside* the range of training errors, which is exactly what the check requires. (b) = 0 sits below every error, so it flags **everything** as an anomaly (all false alarms) and fails the “inside the range” check at the low end. (c) `.max()` sits *at* the very top, so it flags **almost nothing** (misses every anomaly) — the opposite failure, and it too fails the check, which requires the threshold to sit strictly *below* the max. You can raise the percentile for fewer false alarms or lower it to catch more; 99 also passes (still below the max).

*Answers self-check in the notebook: if the `assert` cells pass — `LATENT_DIM == 2`, the loss equals `recon_loss + BETA*kl_loss` (a finite scalar, $KL \geq 0$), and the threshold falls strictly between the min and max training error — you chose right. Then read the histograms **honestly**: resistant and normal spectra overlap — reconstruction error is a **QC / novelty** signal, not a resistance test. The payoff is the smooth VAE latent map (vs. the scattered plain-AE one) and the decoded spectrum morph.*